L32 — Depth First Search (DFS) and Comparison with BFS

Unit: 3 | **Chapter:** Graphs | **BT Level:** BT5 | **CO:** CO2, CO3, CO5

Learning Objectives

- Implement DFS recursively and iteratively
- Trace DFS with timestamps (discovery/finish times)
- Detect cycles, topological sort, and strongly connected components using DFS
- Compare DFS and BFS in terms of strategy, complexity, and applications

---

1. DFS Concept

Depth-First Search (DFS) explores as far as possible along each branch before backtracking.

Intuition: Like navigating a maze — keep going forward until stuck, then backtrack.

```
Graph:            DFS from A (choosing alphabetical neighbor):
  A               Visit A → go deep into B → D → F
 / \              Backtrack → E → backtrack → C
B   C             Final order: A, B, D, F, E, C
|  / \
D E   F
```

---

2. Recursive DFS

```
def dfs_recursive(graph, source):
    visited = set()
    order = []
    discovery = {}
    finish = {}
    timer = [0]   # Using list to allow mutation in nested function

    def dfs(u):
        visited.add(u)
        timer[0] += 1
        discovery[u] = timer[0]
        order.append(u)

        for v in sorted(graph.get(u, [])):   # sorted for determinism
            if v not in visited:
                dfs(v)

        timer[0] += 1
```

```
        finish[u] = timer[0]

    for vertex in sorted(graph.keys()):
        if vertex not in visited:
            dfs(vertex)

    return order, discovery, finish


def dfs_single(graph, source):
    """DFS from a single source, returns visited order."""
    visited = set()
    order = []

    def dfs(u):
        visited.add(u)
        order.append(u)
        for v in graph.get(u, []):
            if v not in visited:
                dfs(v)

    dfs(source)
    return order
```

---

3. Iterative DFS (Explicit Stack)

```
def dfs_iterative(graph, source):
    """Iterative DFS using explicit stack."""
    visited = set()
    stack = [source]
    order = []

    while stack:
        u = stack.pop()
        if u in visited:
            continue
        visited.add(u)
        order.append(u)
        # Push neighbors in reverse order for same traversal as recursi
        for v in reversed(graph.get(u, [])):
            if v not in visited:
                stack.append(v)

    return order
```

---

4. Detailed Trace with Timestamps

Graph: A→B, A→C, B→D, B→E, C→F

```
DFS from A (recursive, alphabetical order):
```

```
Enter A: d[A]=1
  Enter B: d[B]=2
    Enter D: d[D]=3, no unvisited neighbors, f[D]=4
    Enter E: d[E]=5, no unvisited neighbors, f[E]=6
  f[B]=7
  Enter C: d[C]=8
    Enter F: d[F]=9, no unvisited neighbors, f[F]=10
  f[C]=11
f[A]=12
```

Vertex	Discovery	Finish
A	1	12
B	2	7
C	8	11
D	3	4
E	5	6
F	9	10

DFS tree (parenthesis structure):

```
(A (B (D)(E))(C (F)))
```

Finish time order (largest first): A, C, F, B, E, D — useful for topological sort.

---

5. DFS Applications

5.1 Cycle Detection (Directed Graph)

```
def has_cycle_directed(graph):
    """Detect cycle in directed graph using DFS with 3-color marking."""
    WHITE, GRAY, BLACK = 0, 1, 2
    color = {v: WHITE for v in graph}

    def dfs(u):
        color[u] = GRAY   # Currently being explored
        for v in graph.get(u, []):
            if color[v] == GRAY:
                return True   # Back edge → cycle
            if color[v] == WHITE and dfs(v):
                return True
        color[u] = BLACK  # Fully explored
        return False

    return any(dfs(v) for v in graph if color[v] == WHITE)
```

5.2 Topological Sort

Topological ordering: Linear ordering of vertices such that for every directed edge u→v, u appears before v. Only valid for DAGs.

```
def topological_sort(graph):
    """
    DFS-based topological sort.
    Append to result after fully exploring a vertex (on finish).
    """
    visited = set()
    result = []

    def dfs(u):
        visited.add(u)
        for v in graph.get(u, []):
            if v not in visited:
                dfs(v)
        result.append(u)   # Add on finish

    for vertex in graph:
        if vertex not in visited:
            dfs(vertex)

    result.reverse()  # Reverse finish order = topological order
    return result

# Example: Course prerequisites
graph = {
    'Algorithms': ['DS'],
    'DS': ['Programming'],
    'Programming': [],
    'Math': ['Algorithms'],
}
print(topological_sort(graph))
# → ['Math', 'Algorithms', 'DS', 'Programming']
```

5.3 Connected Components (Undirected)

```
def count_components(graph):
    visited = set()
    components = 0

    def dfs(u):
        visited.add(u)
        for v in graph.get(u, []):
            if v not in visited:
                dfs(v)

    for vertex in graph:
        if vertex not in visited:
            dfs(vertex)
            components += 1

    return components
```

5.4 Finding Articulation Points (Bridges)

A vertex whose removal disconnects the graph — key in network reliability analysis.

```
def find_articulation_points(graph):
    """Find all articulation points using Tarjan's algorithm."""
    visited = set()
    disc = {}
    low = {}
    parent = {}
    art_points = set()
    timer = [0]

    def dfs(u):
        visited.add(u)
        timer[0] += 1
        disc[u] = low[u] = timer[0]
        children = 0

        for v in graph.get(u, []):
            if v not in visited:
                children += 1
                parent[v] = u
                dfs(v)
                low[u] = min(low[u], low[v])
                # Articulation point cases:
                if parent.get(u) is None and children > 1:
                    art_points.add(u)   # Root with >1 children
                if parent.get(u) is not None and low[v] >= disc[u]:
                    art_points.add(u)   # Non-root with no back edge
            elif v != parent.get(u):
                low[u] = min(low[u], disc[v])

    for v in graph:
        if v not in visited:
            parent[v] = None
            dfs(v)

    return art_points
```

---

6. DFS vs BFS Comparison

Feature	DFS	BFS
Data structure	Stack (recursive call stack or explicit)	Queue
Traversal strategy	Go deep, then backtrack	Level by level
Space complexity	O(V) — max depth of call stack	O(V) — max width of level
Shortest path	No (not guaranteed)	Yes (unweighted graphs)
Cycle detection	✓ (back edges)	✓ (cross edges)
Topological sort	✓ (reverse finish order)	✗
Connected components	✓	✓
Bipartite check	✓	✓

Best for	Exploring all paths, puzzles, maze	Shortest distance, level-order

When to use DFS:

- Cycle detection in directed graphs
- Topological sorting
- Strongly connected components (Tarjan / Kosaraju)
- Solving puzzles with backtracking (Sudoku, N-queens)
- Generating DFS tree for network analysis

When to use BFS:

- Shortest path in unweighted graphs
- Web crawlers (level-by-level page crawling)
- Social network distance (degrees of separation)
- GPS/map routing on unweighted networks

---

7. Complexity Analysis

Representation	Time	Space
Adjacency List	$O(V + E)$	$O(V)$
Adjacency Matrix	$O(V^2)$	$O(V)$

Each vertex and each edge is visited exactly once → **$O(V + E)$**.

Summary

- DFS explores as deep as possible using recursion or an explicit stack.
- Discovery/finish timestamps enable cycle detection and topological sort.
- Both DFS and BFS are $O(V + E)$ with adjacency list.
- **DFS:** depth-first, finds cycles/topological order; **BFS:** breadth-first, finds shortest paths.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 22.3 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 5.8 (Springer, 2020)